

Namal University Transport Management System (NUTMS)

Database Design Document

By

1	Altaf Hussain	NUM-BSCS-2024-11
2	Rabia Ashraf	NUM-BSCS-2024-65
3	Najia Nayab	NUM-BSCS-2024-62



Department of Computer Sciences
Namal University
Mianwali, Pakistan

Submission Date : 14th June, 2026

TABLE OF CONTENTS

CHAPTER 1: PROJECT OVERVIEW	3
1.1. INTRODUCTION:	3
1.2. PROBLEM STATEMENT:	3
1.3. PROJECT OBJECTIVES:	3
CHAPTER 2: CONCEPTUAL DATABASE DESIGN.....	5
2.1. ENTITY:	5
2.2. DATA DICTIONARY:	6
2.3. BUSINESS RULES:	9
2.4. ENTITY RELATIONSHIP DIAGRAM:	12
CHAPTER 3: LOGICAL DATABASE DESIGN	13
3.1. RELATIONAL SCHEMA:	13
3.2. FUNCTIONAL DEPENDENCIES:	14
3.3. NORMALIZATION:	15
CHAPTER 4: IMPLEMENTATION	18
4.1. LANGUAGE/Framework:	18
4.2. DATABASE CONNECTIVITY:	18
4.3. STORED PROCEDURES AND FUNCTIONS:	19
4.4. INTERFACES:	24
REFERENCES.....	27
APPENDIX: AI USAGE AND PROMPTS.....	27

CHAPTER 1: PROJECT OVERVIEW

1.1. INTRODUCTION:

Namal University, Mianwali, is a private sector university that provides daily transport services to its students, faculty, and staff for commutes between the main campus, hostels, and nearby city areas. Currently, all transport operations are managed through manual, paper-based processes including route planning, driver assignments, trip scheduling, and reporting which introduce errors, delays, data inconsistency, and lack of transparency across the system.

The Namal University Transport Management System (NUTMS) is a web-based platform designed to digitize and automate these transport operations. This Database Design Document focuses on the database design layer of NUTMS the backbone responsible for storing, managing, and reliably retrieving all transport-related data.

The system targets four distinct user classes: Students, who track buses and receive schedule notifications; Faculty, who additionally request special vehicle bookings; Drivers, who view assigned routes and update trip statuses; and Transport Administrators, who manage all users, vehicles, routes, and schedules through a centralized dashboard.

1.2. PROBLEM STATEMENT:

Namal University currently manages all transport operations through manual, paper-based processes with no centralized database system. Information about routes, schedules, drivers, and vehicles is kept in physical registers and spreadsheets, causing data inconsistency and duplication. Students and faculty have no way to check bus locations, schedule changes, or delays in real time, as communication relies entirely on phone calls. Administrators spend excessive time on manual report preparation and resource assignment, increasing error rates and reducing efficiency.

1.3. PROJECT OBJECTIVES:

General Objective: To design and implement a centralized, fully normalized relational database for NUTMS that automates and digitizes all transport operations at Namal University, enabling reliable data storage, real-time information access, and data-driven decision making.

Specific Objectives:

- **Role-Based Access Control:** Define separate database access levels for Students, Faculty, Drivers, and Administrators so that each user can only view and perform operations their role permits.

- **Route and Schedule Management:** Enable administrators to create, update, and manage bus routes, stop sequences, departure times, and arrival times through structured relational tables with referential integrity.
- **Driver and Vehicle Record Maintenance:** Maintain complete records of all drivers and vehicles including capacity, registration details, license information, and driver-to-schedule assignments.
- **Special Vehicle Booking for Faculty:** Support a faculty booking pipeline from submission and pending status through administrative approval or rejection, with automatic notification triggers.
- **Report Generation Support:** Enable the database to run analytical queries generating daily, weekly, and monthly reports on transport usage, peak hours, fuel consumption, and driver trip completion rates.

1.4. GITHUB REPOSITORY LINK:

<https://github.com/Altaf-Hussain-11/DB-Project-NUTMS-.git>

1.5. RELATION SCHEMA LINK:

https://lucid.app/lucidchart/7e35762f-a38a-41b9-8793-d3048aa2d575/edit?viewport_loc=9609%2C-3075%2C4576%2C2481%2C0_0&invitationId=inv_cf61b5dc-09cd-42cc-b25e-8aa33e2dd932

CHAPTER 2: CONCEPTUAL DATABASE DESIGN

2.1. ENTITY:

The following entities form the core of the NUTMS database. Each represents a distinct real-world object within the university transport domain.

Sr. No	Entity Name	Description
01	User	A USER is a person with a registered account in NUTMS. Every user has exactly one role (Student, Faculty, Driver, or Administrator). USER is the supertype from which all four subtypes are derived. Identified by User_ID (PK).
02	STUDENT	A STUDENT is a subtype of USER enrolled at Namal University. Uses NUTMS to track buses and receive schedule notifications. Stores Registration_No and Residential_Type in addition to inherited USER attributes.
03	FACULTY	A FACULTY member is a subtype of USER who is academic or administrative staff. Can submit special vehicle booking requests. Stores Employee_ID, Department, Designation, and Office_Location.
04	DRIVER	A DRIVER is a subtype of USER who operates university buses. Views assigned routes and updates trip statuses in real time. Stores License_Number, License_Expiry, Hire_Date, and Driver_Status.
05	ADMINISTRATOR	An ADMINISTRATOR is a subtype of USER with full system access. Manages users, routes, schedules, vehicles, bookings, and generates reports. Stores Admin_Level and Office_Location.
06	VEHICLE	A VEHICLE is a physical bus operated by Namal University. Stores Registration_Plate, Seating_Capacity, Make_Model, Operational_Status, and GPS_Device_ID. Identified by Vehicle_ID (PK).
07	ROUTE	A ROUTE defines a named path taken by university buses between stops. Stores Route_Name, Start_Location, End_Location, Total_Distance_km, and Estimated_Duration_min. Identified by Route_ID (PK).
08	STOP	A STOP is a designated physical location where passengers board or alight. Stores Stop_Name only. Sequence number and arrival offset are stored in the ROUTE_STOP junction table.
09	SCHEDULE	A SCHEDULE defines a departure time for a VEHICLE operating on a ROUTE on specified days of the week. References ROUTE and VEHICLE via FK. Stores Departure_Time, Days_Of_Week, and effective date range.

10	TRIP	A TRIP is a single real-world execution of a SCHEDULE on a specific date by a specific VEHICLE and DRIVER. Tracks Trip_Status (Scheduled, In Progress, Completed, Cancelled) and actual Departure_Time.
13	BOOKING	A BOOKING is a FACULTY member's formal request for a special vehicle outside the regular schedule. Stores Destination, Trip_Date, Purpose, Passenger_Count, and Status (Pending, Approved, Rejected).
14	VEHICLE_MAINTENANCE	Records every maintenance event performed on a university vehicle. Stores Maintenance_Type, Description, Maintenance_Date, and Maintenance_Cost. Linked to VEHICLE by FK.
15	FUEL_CONSUMPTION	Logs every fuel refill event for a university vehicle. Tracks Liters, Cost (PKR), and Filled_At timestamp. Supports fuel usage analysis and route efficiency reporting.

2.2. DATA DICTIONARY:

The data dictionary below defines attributes, data types, and constraints for the primary NUTMS entities. The complete data dictionary for all entities. Foreign keys are documented separately in the Relational Schema section. .This may include constraints such as required fields, unique values, minimum or maximum lengths, disallowed values, range or format requirements. For each attribute, provide a clear and concise definition or description. Explain what the attribute represents and its significance within the context of the entity. Foreign Key is not part of data dictionary.

2.2.1 USER:

Sr.No	Name	Data Type	Constraint	Description
01	User_ID	INT	PK, AUTO_INCREMENT, NOT NULL	Unique system-generated identifier for each user.
02	Full_Name	VARCHAR(150)	NOT NULL	Full legal name of the user.
03	Email	VARCHAR(150)	NOT NULL, UNIQUE	University email address used for login and notifications.
04	Password_Hash	VARCHAR(255)	NOT NULL	Bcrypt-hashed password; plain text is never stored.
05	Role	INT	NOT NULL, FK -> ROLE	References the ROLE table to determine user access level.
06	Profile_Photo	VARCHAR(255)	OPTIONAL	File path to the user profile picture.
07	Created_At	DATETIME	NOT NULL, DEFAULT NOW()	Timestamp when the account was created.

2.2.2 STUDENT:

Sr.No	Name	Data Type	Constraint	Description
01	User_ID	INT	PK, FK -> USER, NOT NULL	Foreign key referencing USER. Also serves as primary key (shared PK pattern).
02	Registration_No	VARCHAR(30)	NOT NULL, UNIQUE	Official university registration number, e.g. NUM-BSCS-2024-11.
03	Residential_Type	ENUM	NOT NULL	Hosteler or Day Scholar.

2.2.3 FACULTY:

Sr.No	Name	Data Type	Constraint	Description
01	User_ID	INT	PK, FK -> USER, NOT NULL	Foreign key referencing USER. Also serves as primary key (shared PK pattern).
02	Employee_ID	VARCHAR(20)	NOT NULL, UNIQUE	Official university employee identification number.
03	Department	VARCHAR(100)	NOT NULL	Academic or administrative department of the faculty member.
04	Designation	VARCHAR(100)	NOT NULL	Job title, e.g. Lecturer, Assistant Professor.
05	Office_Location	VARCHAR(100)	OPTIONAL	Physical office location on campus.

2.2.4 DRIVER:

Sr.No	Name	Data Type	Constraint	Description
01	User_ID	INT	PK, FK -> USER, NOT NULL	Foreign key referencing USER. Also serves as primary key (shared PK pattern).
02	License_Number	VARCHAR(30)	NOT NULL, UNIQUE	Government-issued driver license number.
03	License_Expiry	DATE	NOT NULL	Expiry date of the driving license.
04	Hire_Date	DATE	NOT NULL	Date the driver was hired by the university.
05	Emergency_Contact	VARCHAR(100)	OPTIONAL	Emergency contact number for the driver.
06	Driver_Status	ENUM	NOT NULL	Active, On Leave, or Terminated.
07	Experience_Years	INT	NOT NULL	Number of years of driving experience of the driver.

2.2.5 VEHICLE:

Sr.No	Name	Data Type	Constraint	Description
01	Vehicle_ID	INT	PK, AUTO_INCREMENT, NOT NULL	Unique system identifier for each vehicle.
02	Registration_Plate	VARCHAR(15)	NOT NULL, UNIQUE	Official vehicle registration plate number.
03	Make_Model	VARCHAR(80)	NOT NULL	Manufacturer and model of the bus, e.g. Hino 300.
04	Seating_Capacity	INT	NOT NULL, > 0	Maximum number of passengers the bus can carry.
05	Operational_Status	ENUM	NOT NULL	Active, Under Maintenance, or Retired.

2.2.6 ROUTE:

Sr.No	Name	Data Type	Constraint	Description
01	Route_ID	INT	PK, AUTO_INCREMENT, NOT NULL	Unique system identifier for each route.
02	Route_Name	VARCHAR(100)	NOT NULL, UNIQUE	Descriptive route name, e.g. Campus to City Centre.
03	Start_Location	VARCHAR(150)	NOT NULL	Name or address of the route starting point.
04	End_Location	VARCHAR(150)	NOT NULL	Name or address of the terminal point.
05	Total_Distance_km	DECIMAL(6,2)	NOT NULL, > 0	Total route length in kilometres.

2.2.12 BOOKING:

Sr.No	Name	Data Type	Constraint	Description
01	Booking_ID	INT	PK, AUTO_INCREMENT, NOT NULL	Unique identifier for each special vehicle booking request.
02	Destination	VARCHAR(200)	NOT NULL	Intended destination of the special trip.

Sr.No	Name	Data Type	Constraint	Description
03	Trip_Date	DATE	NOT NULL	Requested date for the special vehicle trip.
04	Departure_Time	TIME	NOT NULL	Requested departure time for the trip.
05	Purpose	TEXT	NOT NULL	Reason or justification for the special vehicle request.
06	Passenger_Count	INT	NOT NULL, > 0	Total number of passengers expected on the trip.
07	Status	ENUM	NOT NULL	Pending, Approved, or Rejected.
08	Admin_Comments	TEXT	OPTIONAL	Administrator remarks on approval or rejection.
09	Requested_At	DATETIME	NOT NULL, DEFAULT NOW()	Timestamp when the booking request was submitted.
10	Resolved_At	DATETIME	OPTIONAL	Timestamp when the request was approved or rejected.
11	Requested_BY	INT	FK -> USER, NOT NULL	References the FACULTY USER who submitted the booking request.
12	Reviewed_By	INT	FK -> USER, OPTIONAL	References the ADMINISTRATOR USER who approved or rejected the booking.

2.2.13 VEHICLE_MAINTENANCE:

Sr. No	Name	Data Type	Constraint	Description
01	maintenance_id	INT	PK, AUTO_INCREMENT, NOT NULL	Unique identifier for each vehicle maintenance record.
02	maintenance_type	VARCHAR(100)	NOT NULL	Type of maintenance performed, e.g., Engine Service, Oil Change.
03	description	TEXT	OPTIONAL	Detailed description of the maintenance activity performed.
04	maintenance_date	DATE	NOT NULL	Date on which the maintenance activity was carried out.

05	maintenance_cost	DECIMAL(10,2)	NOT NULL, >= 0	Total cost incurred for the maintenance activity in PKR.
06	Vehicle_ID	INT	FK → VEHICLE, NOT NULL	References the vehicle on which maintenance was performed.

2.2.14 FUEL_CONSUMPTION:

Sr. No	Name	Data Type	Constraint	Description
01	fuel_log_id	INT	PK, AUTO_INCREMENT, NOT NULL	Unique identifier for each fuel consumption record.
02	liters	DECIMAL(8,2)	NOT NULL, > 0	Total amount of fuel filled in liters.
03	cost	DECIMAL(10,2)	NOT NULL, >= 0	Total fuel cost in Pakistani Rupees (PKR).
04	Date	DATETIME	NOT NULL	Date and time when the fuel refill was recorded.
05	vehicle_ID	INT	FK → VEHICLE, NOT NULL	References the vehicle for which fuel was consumed.

2.3. BUSINESS RULES:

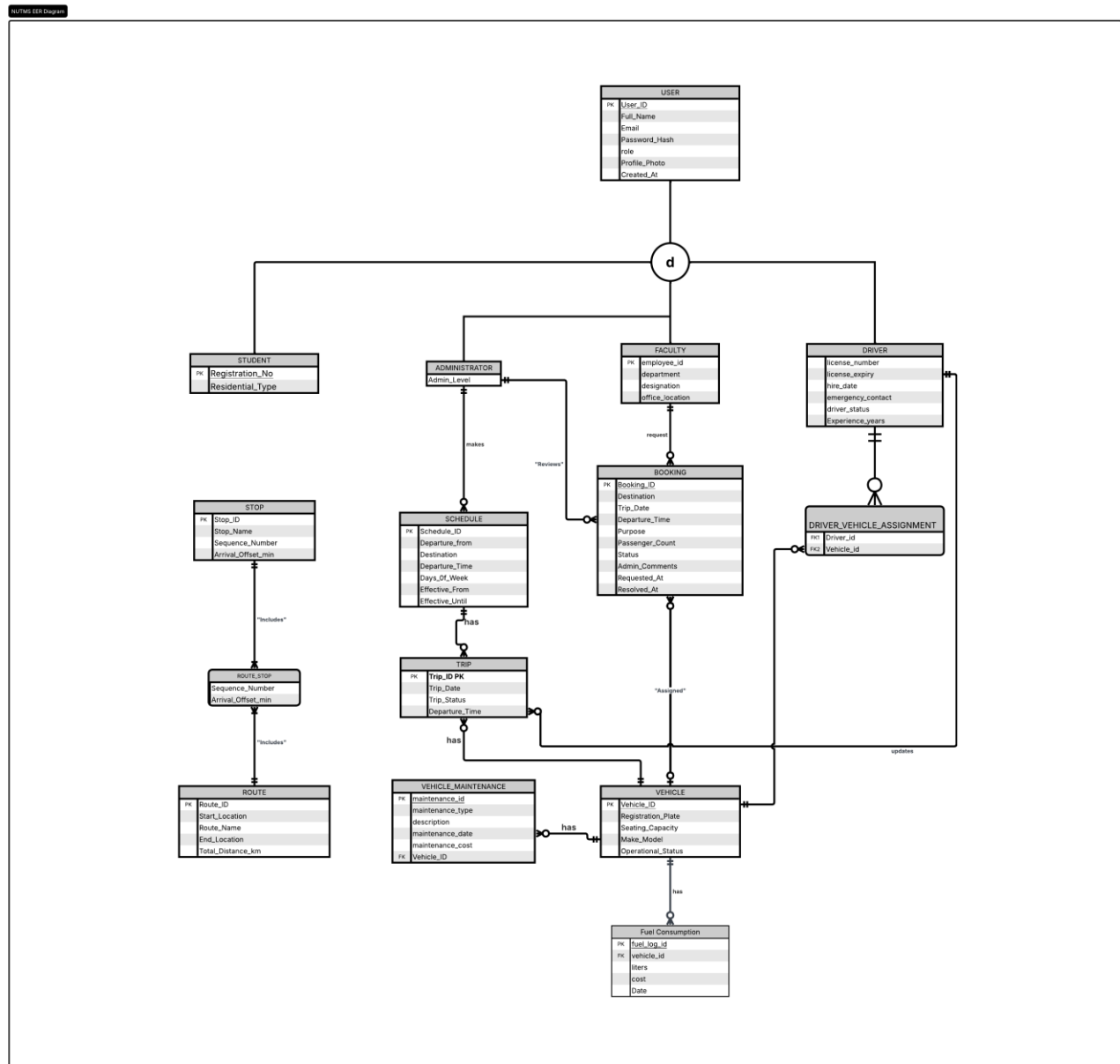
The following business rules govern data integrity, access control, and operational policies in NUTMS. These rules are enforced through SQL constraints, stored procedures, triggers, and application-level validation.

- Every person in NUTMS must be registered as exactly one USER with one of four roles: Student, Faculty, Driver, or Administrator. A USER cannot belong to more than one subtype simultaneously (disjoint specialization). Role changes require administrator action.
- A STUDENT account can only be created by an ADMINISTRATOR; self-registration is not permitted. A USER account may be deactivated by an ADMINISTRATOR
- A DRIVER must have a valid, non-expired driving license.
- A VEHICLE must have Seating_Capacity greater than zero and a unique Registration_Plate. A VEHICLE with Operational_Status of Under Maintenance or Retired cannot be assigned to any new SCHEDULE or TRIP.
- A ROUTE must contain at least two STOPS: a starting stop and a terminal stop. Each stop on a route must have a unique Sequence_Number enforced by a UNIQUE constraint on {Route_ID, Sequence_Number} in ROUTE_STOP.
- A SCHEDULE must reference exactly one ROUTE and one VEHICLE. A VEHICLE cannot have two overlapping SCHEDULEs on the same day and time. Overlap is validated by sp_create_schedule before inserting.

- A DRIVER can be assigned to at most one active TRIP at any given time. A TRIP is generated from a SCHEDULE and must reference the same VEHICLE and DRIVER recorded in that schedule.
- TRIP status must progress forward only: Scheduled -> In Progress -> Completed or Cancelled. Backward transitions are rejected by sp_update_trip_status. A TRIP cannot be set to Completed without an actual Departure_Time being recorded.
- Only FACULTY members may submit special vehicle BOOKING requests
- A BOOKING can only be approved or rejected by an ADMINISTRATOR. An approved BOOKING must have an assigned VEHICLE and DRIVER before the trip date. A BOOKING in Pending status can only be cancelled by the requesting FACULTY member.

2.4. ENTITY RELATIONSHIP DIAGRAM:

The ERD for NUTMS is shown below.



CHAPTER 3: LOGICAL DATABASE DESIGN

3.1. RELATIONAL SCHEMA:

The Enhanced Entity-Relationship Diagram (EERD) of NUTMS was converted into a set of relational tables following standard ERD-to-relational mapping rules. Each strong entity became its own relation with its identifying attribute as the Primary Key (PK). The supertype-subtype hierarchy (USER and its subtypes STUDENT, FACULTY, DRIVER, and ADMINISTRATOR) was mapped using the shared primary key approach, where each subtype table uses User_ID as both its Primary Key and a Foreign Key (FK) referencing USER. One-to-many relationships were represented by placing a foreign key on the "many" side, while the many-to-many relationship between ROUTE and STOP was resolved into the bridge relation ROUTE_STOP, carrying a composite primary key made up of the foreign keys from both participating entities. The resulting relations are listed below, with PK denoting Primary Key and FK denoting Foreign Key.

1. **USER** (User_ID [PK], Full_Name, Email, Password_Hash, role, Profile_Photo, Created_At)
2. **STUDENT** (User_ID [PK, FK → USER], Registration_No, Residential_Type)
3. **FACULTY** (User_ID [PK, FK → USER], Employee_ID, Department, Designation, Office_Location)
4. **DRIVER** (User_ID [PK, FK → USER], License_Number, License_Expiry, Hire_Date, Emergency_Contact, Driver_Status, Experience_Years)
5. **ADMINISTRATOR** (User_ID [PK, FK → USER], Admin_Level)
6. **VEHICLE** (Vehicle_ID [PK], Registration_Plate, Seating_Capacity, Make_Model, Operational_Status, GPS_Device_ID [FK → GPS])
7. **ROUTE** (Route_ID [PK], Start_Location, Route_Name, End_Location, Total_Distance_km, Estimated_Duration_min)
8. **STOP** (Stop_ID [PK], Stop_Name, Sequence_Number, Arrival_Offset_min)
9. **ROUTE_STOP** (Stop_ID [PK, FK → STOP], Route_ID [PK, FK → ROUTE], Sequence_Number, Arrival_Offset_min) — bridge relation resolving the many-to-many relationship between ROUTE and STOP.
10. **SCHEDULE** (Schedule_ID [PK], Departure_from, Destination, Departure_Time, Days_Of_Week, Effective_From, Effective_Until, Admin_ID [FK → ADMINISTRATOR])
11. **DRIVER_VEHICLE_ASSIGNMENT** (Driver_id [FK → DRIVER], Vehicle_id [FK → VEHICLE])
12. **TRIP** (Trip_ID [PK], Trip_Date, Trip_Status, Departure_Time, Driver_ID [FK → DRIVER], Schedule_ID [FK → SCHEDULE], Vehicle_ID [FK → VEHICLE])
13. **GPS** (Log_ID [PK], Timestamp, Latitude, Longitude, Speed_kmph, Vehicle_ID [FK → VEHICLE])
14. **BOOKING** (Booking_ID [PK], Destination, Trip_Date, Departure_Time, Purpose, Passenger_Count, Status, Admin_Comments, Requested_At, Resolved_At, Requested_By [FK → FACULTY], Reviewed_By [FK → ADMINISTRATOR])
15. **VEHICLE_MAINTENANCE** (maintenance_id [PK], maintenance_type, description, maintenance_date, maintenance_cost, Vehicle_ID [FK → VEHICLE])

16. **FUEL_CONSUMPTION** (fuel_log_id [PK], liters, cost, petrol_pump, Date, vehicle_id [FK → VEHICLE])

3.2. FUNCTIONAL DEPENDENCIES:

The notation A -> B indicates that attribute A functionally determines attribute B.

3.2.1 USER

- User_ID -> Full_Name, Email, Password_Hash, Role, Profile_Photo, Created_At
- Email -> User_ID

3.2.2 STUDENT

- User_ID -> User_ID, Residential_Type
- Registration_No -> User_ID

3.2.3 FACULTY

- User_ID -> Employee_ID, Department, Designation, Office_Location
- Employee_ID -> User_ID

3.2.4 DRIVER

- User_ID -> License_Number, License_Expiry, Hire_Date, Emergency_Contact, Salary, Driver_Status, Experience_Years
- License_Number -> User_ID

3.2.5 ADMINISTRATOR

- User_ID -> Admin_Level

3.2.6 VEHICLE

- Vehicle_ID -> Registration_Plate, Seating_Capacity, Make_Model, Operational_Status, GPS_Device_ID
- Registration_Plate -> Vehicle_ID
- GPS_Device_ID -> Vehicle_ID

3.2.7 ROUTE

- Route_ID -> Start_Location, Route_Name, End_Location, Total_Distance_km, Estimated_Duration_min

3.2.8 STOP

- Stop_ID -> Stop_Name, Sequence_Number, Arrival_Offset_min

3.2.9 ROUTE_STOP

- {Route_ID, Stop_ID} -> Sequence_Number, Arrival_Offset_min

Both attributes are fully dependent on the composite primary key. Neither Route_ID alone nor Stop_ID alone determines these attributes independently.

3.2.10 SCHEDULE

- Schedule_ID -> Route_ID, Vehicle_ID, Departure_From, Destination, Departure_Time, Days_Of_Week, Effective_From, Effective_Until

3.2.11 TRIP

- Trip_ID -> Schedule_ID, Trip_Date, Trip_Status, Departure_Time

3.2.12 BOOKING

- Booking_ID -> Destination, Trip_Date, Departure_Time, Purpose, Passenger_Count, Status, Admin_Comments, Requested_At, Resolved_At, Requested_By, Reviewed_By

3.2.13 GPS_LOG

- Log_ID -> Vehicle_ID, Timestamp, Latitude, Longitude, Speed_kmph
- {Vehicle_ID, Timestamp} -> Latitude, Longitude, Speed_kmph (natural candidate key)

3.2.14 VEHICLE_MAINTENANCE

- Maintenance_ID -> Vehicle_ID, Maintenance_Type, Description, Maintenance_Date, Maintenance_Cost

3.2.15 FUEL_CONSUMPTION

- Fuel_Log_ID -> Vehicle_ID, Liters, Cost, Petrol_Pump, Date

3.3. NORMALIZATION:

This section explains how the NUTMS relations were normalized step by step to 1NF, 2NF, and 3NF, removing repeating groups, partial dependencies, and transitive dependencies along the way.

1NF — Removing Repeating Groups

A relation is in 1NF if every attribute holds a single (atomic) value and a primary key is defined.

- **SCHEDULE.Days_Of_Week**, originally stored multiple values together (e.g., "Mon, Tue, Wed, Thu, Fri"), which is a repeating group. This was fixed by storing it as a MySQL SET, where each day is a separate value.

All other relations already had atomic attributes and a clear primary key, so they satisfied 1NF without changes.

2NF — Removing Partial Dependencies

A relation is in 2NF if it is in 1NF and every non-key attribute depends on the whole primary key, not just part of it. This only matters for relations with composite keys.

- **ROUTE_STOP** has the composite key {Route_ID, Stop_ID}. Its attributes (Sequence_Number, Arrival_Offset_min) depend on both parts together, so no partial dependency exists — it is already in 2NF.
- **STOP** originally also stored Sequence_Number and Arrival_Offset_min. But these values only make sense for a stop within a specific route, not for the stop on its own. This was a hidden partial dependency, so these two attributes were removed from STOP and kept only in ROUTE_STOP, where they correctly depend on the full composite key.

All other relations have single-column primary keys, so they are automatically in 2NF.

3NF — Removing Transitive Dependencies

A relation is in 3NF if it is in 2NF and no non-key attribute depends on another non-key attribute (instead of depending only on the primary key).

- **SCHEDULE** originally stored Departure_from and Destination directly, but these are really properties of a ROUTE, not of the schedule itself. This created a transitive dependency (Schedule_ID → Route_ID → Start_Location/End_Location). Fix: remove Departure_from and Destination from SCHEDULE and add a Route_ID foreign key to ROUTE instead.
- **VEHICLE.GPS_Device_ID** was originally linked as a foreign key to the GPS table, while GPS also linked back to VEHICLE — creating a circular dependency. Fix: keep GPS_Device_ID in VEHICLE only as a descriptive attribute (the hardware ID), and let the GPS table's Vehicle_ID be the only link between the two.
- **BOOKING** stores Requested_By and Reviewed_By as foreign keys only, with no extra details about the requester/reviewer stored alongside — so no transitive dependency exists here.

After these changes, every relation in NUTMS satisfies 1NF, 2NF, and 3NF.

Final 3NF Relations

USER (User_ID [PK], Full_Name, Email, Password_Hash, role, Profile_Photo, Created_At)

STUDENT (User_ID [PK, FK → USER], Registration_No, Residential_Type)

FACULTY (User_ID [PK, FK → USER], Employee_ID, Department, Designation, Office_Location)

DRIVER (User_ID [PK, FK → USER], License_Number, License_Expiry, Hire_Date, Emergency_Contact, Driver_Status, Experience_Years)

ADMINISTRATOR (User_ID [PK, FK → USER], Admin_Level)

VEHICLE (Vehicle_ID [PK], Registration_Plate, Seating_Capacity, Make_Model, Operational_Status, GPS_Device_ID)

ROUTE (Route_ID [PK], Start_Location, Route_Name, End_Location, Total_Distance_km, Estimated_Duration_min)

STOP (Stop_ID [PK], Stop_Name)

ROUTE_STOP (Route_ID [PK, FK → ROUTE], Stop_ID [PK, FK → STOP], Sequence_Number, Arrival_Offset_min)

SCHEDULE (Schedule_ID [PK], Route_ID [FK → ROUTE], Departure_Time, Days_Of_Week, Effective_From, Effective_Until, Admin_ID [FK → ADMINISTRATOR])

DRIVER_VEHICLE_ASSIGNMENT (Assignment_ID [PK], Driver_id [FK → DRIVER], Vehicle_id [FK → VEHICLE])

TRIP (Trip_ID [PK], Trip_Date, Trip_Status, Departure_Time, Driver_ID [FK → DRIVER], Schedule_ID [FK → SCHEDULE], Vehicle_ID [FK → VEHICLE])

GPS (Log_ID [PK], Timestamp, Latitude, Longitude, Speed_kmph, Vehicle_ID [FK → VEHICLE])

BOOKING (Booking_ID [PK], Destination, Trip_Date, Departure_Time, Purpose, Passenger_Count, Status, Admin_Comments, Requested_At, Resolved_At, Requested_By [FK → FACULTY], Reviewed_By [FK → ADMINISTRATOR])

VEHICLE_MAINTENANCE (maintenance_id [PK], maintenance_type, description, maintenance_date, maintenance_cost, Vehicle_ID [FK → VEHICLE])

FUEL_CONSUMPTION (fuel_log_id [PK], liters, cost, petrol_pump, Date, vehicle_id [FK → VEHICLE])

CHAPTER 4: IMPLEMENTATION

4.1. LANGUAGE/FRAMEWORK:

Frontend

The frontend of the Namal University Transport Management System (NUTMS) was developed using **React.js with TypeScript**. React's component-based architecture allows the development of reusable and modular user interface components across all user roles, including Administrator, Faculty, Driver, and Student. TypeScript provides static type checking, reducing runtime errors and improving code maintainability. Additionally, **Vite** was used as the build tool to ensure fast development and optimized production builds, while **React Router DOM** manages client-side navigation without requiring full page reloads.

Backend

The backend was implemented using **Node.js** and the **Express.js** framework. Node.js provides a non-blocking, event-driven architecture suitable for handling multiple concurrent user requests. Express.js simplifies API development through efficient route management, middleware integration, and error handling. For real-time bus tracking and GPS monitoring, **Socket.io** was integrated to facilitate instant communication between clients and the server.

Justification of Selected Technology Stack

The selected technology stack offers a unified JavaScript/TypeScript development environment across both frontend and backend. React ensures efficient user interface rendering through its virtual DOM, while Node.js effectively manages high-volume API requests. This combination is particularly suitable for a real-time transport management system where performance, scalability, and maintainability are critical requirements.

4.2. DATABASE CONNECTIVITY:

Database Management System

The NUTMS uses **MySQL 9.7 Community Edition** as its primary database management system.

Database Connectivity Library

The application uses the **mysql2/promise** library, which supports asynchronous database operations through async/await syntax and prepared statements.

Database Connection Architecture

React Frontend (Port 5173)

↓ HTTP/REST Requests (Axios)

Node.js / Express API (Port 5000)

↓ mysql2/promise Connection Pool

MySQL Database Server (Port 3306)

Connection Parameters

Parameter	Value
Database Host	localhost
Database User	root
Database Name	NUTMS
Database Port	3306
Connection Pool Size	10

Connection Pool Management

A pool of ten persistent database connections is maintained using `mysql2/promise.createPool()`. This approach minimizes connection overhead and improves system performance. All database operations are performed exclusively through stored procedures using helper functions such as `callProc()` and `callProcSingle()`. No raw SQL queries are executed directly within the application layer.

Authentication Mechanism

The system employs **JSON Web Tokens (JWT)** for securing API endpoints. Upon successful login, users receive a signed token which must be included in subsequent requests using the Authorization header. Backend middleware validates the token before granting access to protected resources.

4.3. STORED PROCEDURES AND FUNCTIONS:

5. The database layer relies heavily on stored procedures, functions, and triggers to ensure data integrity, modularity, and maintainability.

#	Object Name	Type	Description / Purpose	Input Parameters	Output / Return Value
---	-------------	------	-----------------------	------------------	-----------------------

1	sp_GetUserByEmail	Procedure	Returns user details by email address.	p_Email VARCHAR(150)	User_ID, Full_Name, Email, Password_Hash, role_id, Created_At
2	sp_AddUser	Procedure	Creates a new USER record and validates email uniqueness.	p_FullName, p_Email, p_PasswordHash, p_Role	New_User_ID
3	sp_UpdateUser	Procedure	Updates USER information and logs changes.	p_UserID, p_FullName, p_Email	Rows affected
4	sp_DeleteUser	Procedure	Deletes/deactivates a user account.	p_UserID INT	Rows affected
5	sp_GetUsers	Procedure	Returns all registered users.	p_Role VARCHAR(20)	User_ID, Full_Name, Email, role_id, Created_At
6	sp_GetUserProfile	Procedure	Returns complete profile information.	p_UserID INT	User_ID, Full_Name, Email, role_id, Created_At
7	sp_UpdateProfile	Procedure	Updates profile information.	p_UserID, p_FullName, p_Email	Rows affected
8	sp_ChangePassword	Procedure	Changes user password.	p_UserID INT, p_PasswordHash VARCHAR(255)	Rows affected
9	sp_AddStudent	Procedure	Creates a student record.	p_FullName, p_Email, p_PasswordHash, p_RegNo, p_ResType	None
10	sp_GetStudents	Procedure	Returns all student records.	None	Registration_No, User_ID, Full_Name, Email, Residential_Type
11	sp_UpdateStudent	Procedure	Updates student information.	p_RegNo, p_FullName,	Rows affected

				p_Email, p_ResType	
12	sp_DeleteStudent	Procedure	Deletes student record.	p_RegNo VARCHAR(30)	Rows affected
13	sp_AddVehicle	Procedure	Adds a new vehicle.	p_Plate, p_Capacity, p_MakeModel , p_Status, p_GPS	None
14	sp_GetVehicles	Procedure	Returns all vehicles.	None	All VEHICLE columns
15	sp_UpdateVehicleStatus	Procedure	Updates vehicle operational status.	p_VehicleID INT, p_Status ENUM	Rows affected
16	sp_DeleteVehicle	Procedure	Deletes vehicle record.	p_VehicleID INT	Rows affected
17	sp_AddRoute	Procedure	Creates a new route.	p_StartLoc, p_RouteName, p_EndLoc, p_Distance, p_Duration	None
18	sp_GetRoutes	Procedure	Returns all routes.	None	All ROUTE columns
19	sp_DeleteRoute	Procedure	Deletes a route.	p_RouteID INT	Rows affected
20	sp_GetRouteStops	Procedure	Returns stops of a route.	p_RouteID INT	Route_ID, Route_Name, Stop_ID, Stop_Name, Sequence_Number, Arrival_Offset_min
21	sp_AddSchedule	Procedure	Creates a schedule.	p_From, p_To, p_DepartureTime, p_Days, p_EffectiveFrom, p_RouteID, p_AdminID	None
22	sp_GetSchedules	Procedure	Returns all schedules.	None	Schedule_ID, Departure_from, Destination, Departure_Time, Days_Of_Wee

					k, Route Name
23	sp_DeleteSchedule	Procedure	Deletes a schedule.	p_ScheduleID INT	Rows affected
24	sp_AddTrip	Procedure	Creates a trip.	p_TripDate, p_ScheduleID, p_VehicleID	None
25	sp_UpdateTripStatus	Procedure	Updates trip status.	p_TripID INT, p_Status ENUM	Rows affected
26	sp_GetTrips	Procedure	Returns all trips.	p_Status VARCHAR(20)	Trip_ID, Trip_Date, Trip_Status, Departure_from, Destination, Registration_Plate
27	sp_DeleteTrip	Procedure	Deletes a trip.	p_TripID INT	Rows affected
28	sp_AddBooking	Procedure	Creates a booking request.	p_Destination, p_TripDate, p_DepartureTime, p_Purpose, p_PassengerCount, p_EmployeeID	None
29	sp_GetBookings	Procedure	Returns all booking requests.	p_Status VARCHAR(20)	Booking_ID, Destination, Trip_Date, Status, Requested_By
30	sp_ResolveBooking	Procedure	Approves or rejects a booking request.	p_BookingID, p_Status, p_Comments	Rows affected
31	sp_DeleteBooking	Procedure	Deletes a booking request.	p_BookingID INT	Rows affected
32	sp_GetDrivers	Procedure	Returns all drivers.	None	driver_id, Full_Name, Email, license_number, driver_status
33	sp_UpdateDriverStatus	Procedure	Updates driver status.	p_DriverID VARCHAR(20), p_Status ENUM	Rows affected

34	sp_AssignDriverToVehicle	Procedure	Assigns a driver to a vehicle.	p_DriverID, p_VehicleID, p_AdminID	None
35	sp_GetAssignments	Procedure	Returns all assignments.	None	Assignment_id, Driver_Name, Registration_Plate, Assignment_status
36	sp_EndAssignment	Procedure	Ends an active assignment.	p_AssignmentID INT	Rows affected
37	sp_AddGPSLog	Procedure	Stores GPS coordinates.	p_Lat, p_Lng, p_Speed, p_VehicleID	None
38	sp_GetLatestGPS	Procedure	Returns latest GPS location.	None	Vehicle_ID, Registration_Plate, Latitude, Longitude, Speed_kmph, Timestamp
39	sp_AddMaintenance	Procedure	Creates maintenance record.	p_Type, p_Description, p_Date, p_Cost, p_VehicleID	None
40	sp_GetMaintenanceByVehicle	Procedure	Returns maintenance history.	p_VehicleID INT	All VEHICLE_MAINTENANCE columns
41	fn_TotalMaintenanceCost	Function	Calculates total maintenance expenditure for a vehicle.	p_VehicleID INT	DECIMAL(10,2) total cost
42	fn_PendingBookingsCount	Function	Returns the number of pending booking requests.	None	INT count
43	fn_ActiveVehicleCount	Function	Returns the total number of active vehicles.	None	INT count
44	trg_Trip_DepartureStamp	Trigger	Records departure time when trip becomes In Progress.	Fires on TRIP UPDATE	Sets NEW.Departure_Time = NOW()

45	trg_Booking_Resolved Stamp	Trigger	Records resolution time when booking is approved/rejected.	Fires on BOOKING UPDATE	Sets NEW.Resolved_At = NOW()
46	trg_Maintenance_VehicleStatus	Trigger	Sets vehicle status to Under Maintenance when maintenance is inserted.	Fires on VEHICLE_MAINTENANCE INSERT	Updates VEHICLE.Operational_Status
47	trg_Assignment_Close Existing	Trigger	Closes existing assignment before creating a new one.	Fires on DRIVER_VE_ASSIGNMENT INSERT	Updates old assignment to Completed

5.1. INTERFACES:

The graphical user interfaces developed for the NUTMS provide role-specific functionality to administrators, faculty members, drivers, and students. Screenshots of the following interfaces should be included in the report along with a brief description beneath each image.

Screens to Include

Landing Page

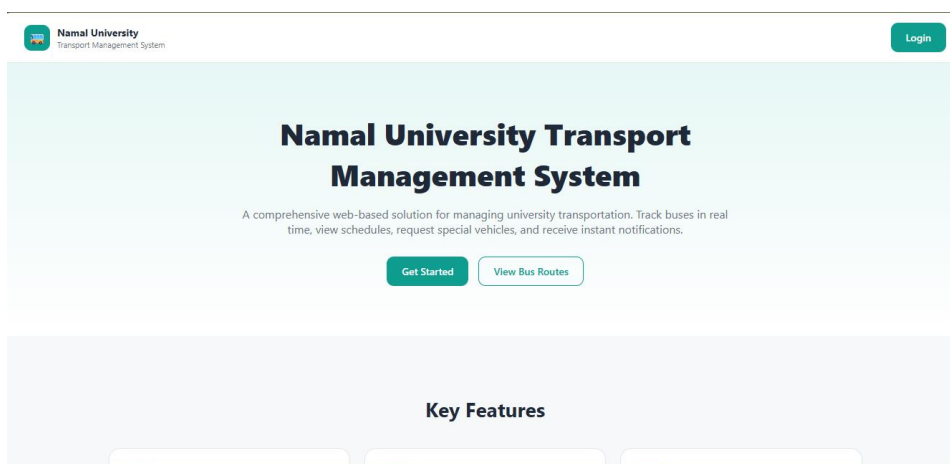


Figure 5.1: Landing Page of the University Transport Management System

Login Page

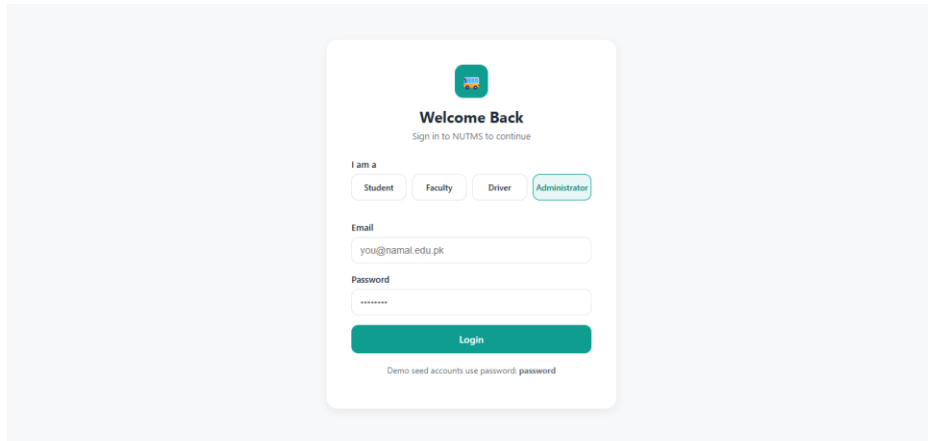


Figure 6.2: User Login Page

Administrator Dashboard

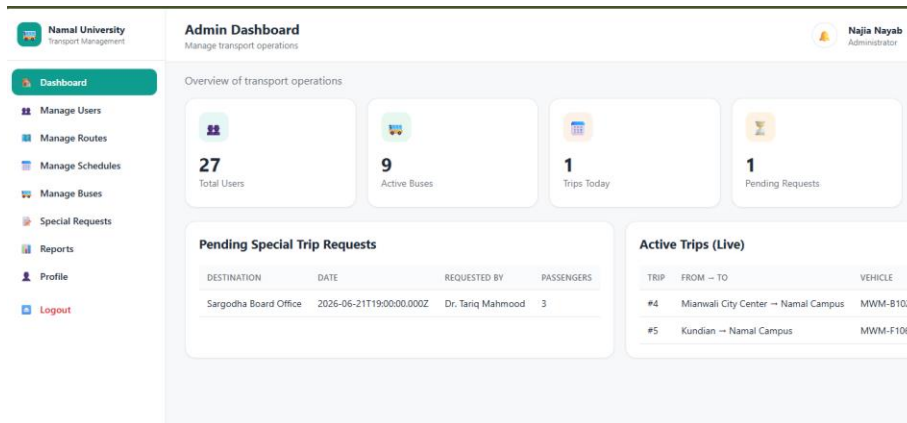


Figure 5.3: Administrator Dashboard

Faculty Dashboard

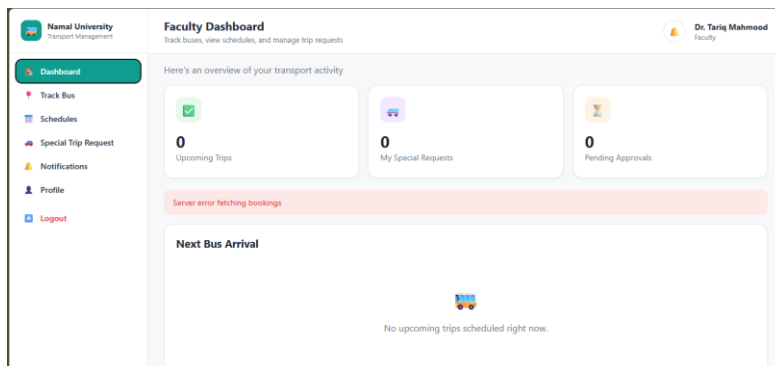


Figure 5.4: Faculty Dashboard

Driver Dashboard

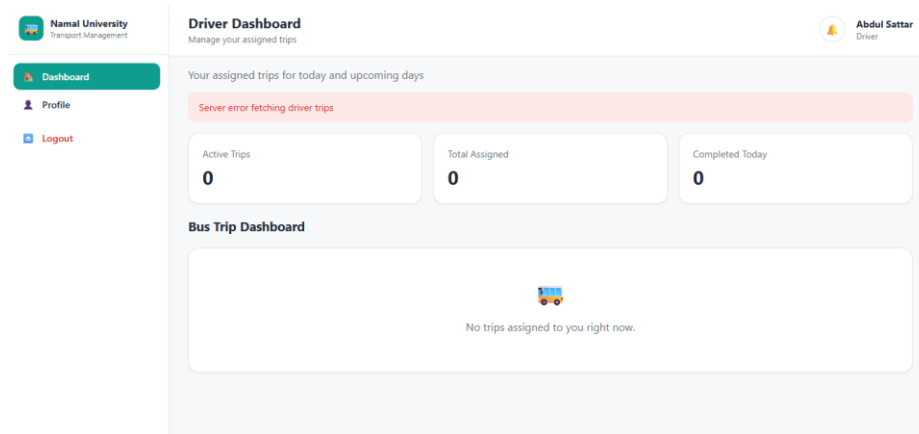


Figure 5.5: Driver Dashboard

Student Dashboard

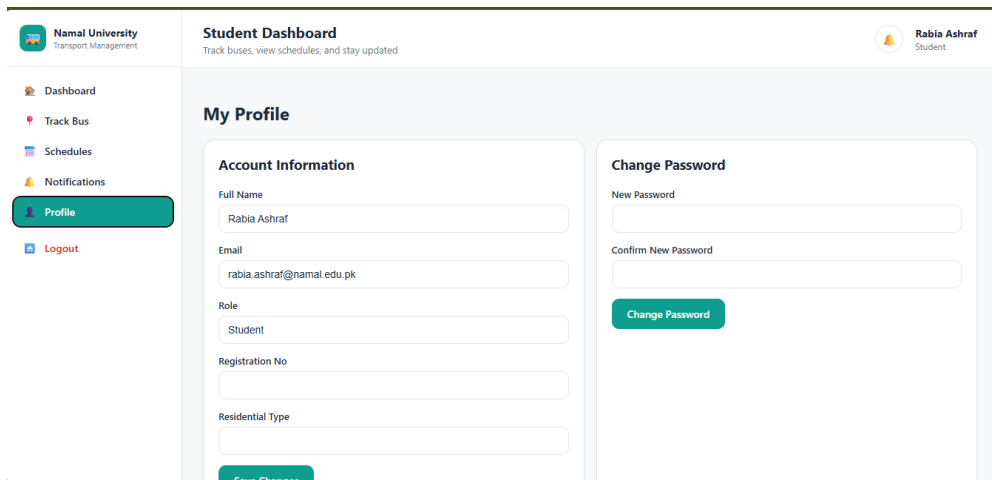


Figure 6.6: Student Dashboard

APPENDIX: AI USAGE AND PROMPTS

The following AI tools were used during Milestone 3. All AI-generated content was reviewed, verified, and personalized before inclusion.

AI Tool	Purpose	Usage Description	Date
Claude	ERD analysis, entity/attribute identification, relational schema, normalization documentation, business rules, template report generation	Uploaded Milestone 2 report, ERD image, and template. Prompted to Analyze the NUTMS ERD and indicate all mistakes including wrong/missing relationships and missing entities. Then Generated a complete Milestone 3 database design report filling the provided template with NUTMS content, then this generated report was settled according to Project Requirments.	June 2026
Claude	DDL and DML script generation	Generated complete MySQL DDL script (dbDDL.sql) for the NUTMS schema with all tables, constraints, indexes, views, stored procedures, functions, and triggers. Generated DML script (dbDML.sql) with at least 20 meaningful sample rows per table.	June 2026
Claude & ChatGPT	GUI	Generated the Code for gui from different AI tools and took help to integrate them with the Database	June 2026
LUCID AI & ChatGPT	Relation Schema	Took Help from the Chatgpt to make the Relation Schema then took help from Lucid ai to arrange the Schema	June 2026